

	Cairo University Faculty of Graduate Studies for Statistical Research Software Engineering Program	
--	---	--

Professional Master's Graduation Project

Software Engineering Program – Coursework Track

Project Title

[Insert the full project title here]

Student Name	[Insert student full name]
Student ID	[Insert student ID]
Program	Master of Software Engineering
Track	Coursework Track
Supervisor Name	[Insert supervisor full name]
Academic Year	[20XX / 20XX]
Submission Date	[Month Day, Year]

Submitted in partial fulfillment of the requirements for the Professional Master's Degree in Software Engineering.

Document Control

0.1	[Date]	[Name]	[Insert notes]

How to Use This Template

Replace all text inside square brackets [like this] with project-specific content.

Keep the section order unless the supervisor or program coordinator approves a change.

Write in clear academic English and support claims with citations.

Use figures, tables, diagrams, and references where they improve clarity.

Delete instructional notes before final submission.

Recommended Formatting

Paper size	A4
Font	Arial or Times New Roman, 11–12 pt
Line spacing	1.15 or 1.5
Margins	Normal academic margins, approx. 2–2.5 cm
Citation style	APA, IEEE, or department-approved style
Expected length	As defined by the program coordinator / supervisor

Table of Contents

[Insert an automatic Table of Contents here before final submission: References > Table of Contents in Microsoft Word.]

Suggested structure:

Abstract

1. Introduction

2. Problem Definition

3. Existing Solution Approaches

4. Proposed Solution

5. System Analysis and Design

6. Implementation

7. Testing and Evaluation

8. Discussion After Applying the Solution

9. Conclusion and Future Work

References

Appendices

Abstract

Purpose

[Summarize the project purpose, the problem addressed, the proposed solution, the implemented system or prototype, and the key outcomes.]

Guidance

[Keep the abstract concise. Mention the problem domain, methodology, tools/technologies, evaluation approach, and main result. Avoid detailed background.]

Keywords

Guidance

[List 4–6 keywords separated by commas, e.g., Software Engineering, Web Application, Machine Learning, Requirements Engineering, API Integration.]

1. Introduction

1.1 Background

[Introduce the general domain and context of the project. Explain why the area is important from an academic, industrial, or practical perspective.]

1.2 Project Motivation

[Explain why this project is worth doing. Describe the business, technical, operational, educational, or societal motivation.]

1.3 Project Objectives

[State the main objectives as clear, measurable points.]

[Objective 1: Insert a specific and measurable objective.]

[Objective 2: Insert a specific and measurable objective.]

[Objective 3: Insert a specific and measurable objective.]

[Objective 4: Insert a specific and measurable objective.]

1.4 Project Scope

[Define what is included and what is not included in the project. Mention assumptions, constraints, and boundaries.]

1.5 Document Organization

[Briefly describe what each chapter contains.]

2. Problem Definition

2.1 Problem Statement

[Clearly define the problem in one or two paragraphs. The reader should understand what is wrong, missing, inefficient, risky, or costly in the current situation.]

2.2 Stakeholders and Users

[Identify the target users, system actors, and stakeholders affected by the problem.]

2.3 Current Situation / As-Is Process

[Describe how the problem is currently handled. Include workflow diagrams, screenshots, or examples if applicable.]

2.4 Problem Impact

[Explain the consequences of not solving the problem. Discuss cost, time, quality, usability, accuracy, scalability, security, or maintainability impact.]

2.5 Requirements Derived from the Problem

[List the high-level functional and non-functional requirements that come directly from the problem definition.]

3. Existing Solution Approaches

3.1 Literature / Market Review

[Review relevant academic work, tools, systems, frameworks, or commercial solutions related to the problem.]

3.2 Existing Methods and Techniques

[Explain the major methods currently used to solve similar problems. Compare algorithmic, architectural, or process alternatives where relevant.]

3.3 Comparative Analysis

[Compare existing approaches using criteria such as cost, complexity, accuracy, scalability, maintainability, usability, and security.]

Criterion	Current / Existing Approach	Proposed Solution	Comment
[Criterion]	[Description]	[Description]	[Analysis]
[Criterion]	[Description]	[Description]	[Analysis]
[Criterion]	[Description]	[Description]	[Analysis]
[Criterion]	[Description]	[Description]	[Analysis]

3.4 Limitations of Existing Solutions

[Identify gaps and limitations that justify the need for the student's proposed solution.]

4. Proposed Solution

4.1 Solution Overview

[Describe the proposed solution in clear terms. Explain what will be built, improved, automated, analyzed, or integrated.]

4.2 Solution Rationale

[Explain why this solution is suitable compared with the alternatives discussed in Chapter 3.]

4.3 Key Features

[List the major system features and explain how each one contributes to solving the problem.]

4.4 Proposed Architecture

[Describe the high-level architecture. Include a diagram showing major components, data flow,

integrations, and deployment view where applicable.]

[Insert diagram / figure here]

Diagram / Figure Placeholder

4.5 Technology Stack

[List the selected programming languages, frameworks, databases, tools, cloud services, APIs, or libraries, and justify each selection.]

4.6 Risks and Constraints

[Discuss implementation risks, technical constraints, data limitations, security considerations, and mitigation actions.]

5. System Analysis and Design

5.1 Functional Requirements

[Provide detailed functional requirements, preferably numbered, testable, and linked to user needs.]

ID	Requirement	Priority	Source / Rationale
REQ-001	[Insert requirement]	[High/Medium/Low]	[Link to problem/user need]
REQ-002	[Insert requirement]	[High/Medium/Low]	[Link to problem/user need]
REQ-003	[Insert requirement]	[High/Medium/Low]	[Link to problem/user need]

5.2 Non-Functional Requirements

[Cover performance, security, usability, reliability, maintainability, compatibility, scalability, and availability requirements.]

ID	Requirement	Priority	Source / Rationale
REQ-001	[Insert requirement]	[High/Medium/Low]	[Link to problem/user need]
REQ-002	[Insert requirement]	[High/Medium/Low]	[Link to problem/user need]
REQ-003	[Insert requirement]	[High/Medium/Low]	[Link to problem/user need]

5.3 Use Case Model / User Stories

[Provide use cases or user stories with actors, preconditions, main flow, alternative flows, and postconditions.]

5.4 Data Model

[Describe the database schema, entities, relationships, and key data fields. Include ERD if applicable.]

[Insert diagram / figure here]

Diagram / Figure Placeholder

5.5 User Interface Design

[Include wireframes, screen descriptions, navigation flow, and usability considerations.]

[Insert diagram / figure here]

Diagram / Figure Placeholder

5.6 System Design Diagrams

[Include relevant UML or architecture diagrams such as class diagram, sequence diagram, component diagram, deployment diagram, or activity diagram.]

[Insert diagram / figure here]

Diagram / Figure Placeholder

6. Implementation

6.1 Development Environment

[Describe the environment used for development: IDE, operating system, database, server, libraries, and version control.]

6.2 Implementation Details

[Explain the main modules, components, algorithms, APIs, database access, business logic, and integration points.]

6.3 Important Code / Configuration Decisions

[Discuss important implementation choices without pasting excessive code. Place long code listings in appendices.]

6.4 Security and Data Protection

[Explain authentication, authorization, validation, encryption, secure configuration, data privacy, and error handling where applicable.]

6.5 Deployment / Execution Instructions

[Describe how to install, configure, run, and deploy the system or prototype.]

7. Testing and Evaluation

7.1 Testing Strategy

[Describe the testing levels used: unit testing, integration testing, system testing, user acceptance testing, performance testing, or security testing.]

7.2 Test Cases

[Provide representative test cases with expected results and actual results.]

Test Case ID	Scenario	Expected Result	Actual Result	Status
TC-001	[Insert scenario]	[Expected]	[Actual]	[Pass/Fail]
TC-002	[Insert scenario]	[Expected]	[Actual]	[Pass/Fail]
TC-003	[Insert scenario]	[Expected]	[Actual]	[Pass/Fail]
TC-004	[Insert scenario]	[Expected]	[Actual]	[Pass/Fail]

7.3 Evaluation Metrics

[Define how success is measured. Examples: accuracy, response time, completion time, error rate, user satisfaction, maintainability, or throughput.]

7.4 Results

[Present test results using tables, charts, screenshots, and concise analysis.]

7.5 Validation Against Objectives

[Explain whether the project objectives defined in Chapter 1 were achieved.]

[Objective 1: Insert a specific and measurable objective.]

[Objective 2: Insert a specific and measurable objective.]

[Objective 3: Insert a specific and measurable objective.]

[Objective 4: Insert a specific and measurable objective.]

8. Discussion After Applying the Solution

8.1 Problem Status After Solution

[Discuss how the proposed solution changed the original problem situation. Refer back to the problem definition and objectives.]

8.2 Benefits Achieved

[Describe practical, technical, operational, academic, or business benefits achieved by the solution.]

8.3 Remaining Limitations

[State what the solution still does not solve. Be honest about scope limitations, data limitations, technical constraints, and assumptions.]

8.4 Lessons Learned

[Discuss lessons learned during analysis, design, implementation, testing, project management, and teamwork.]

8.5 Comparison: Before vs. After

[Compare the situation before and after the proposed solution using measurable or observable criteria.]

Criterion	Current / Existing Approach	Proposed Solution	Comment
[Criterion]	[Description]	[Description]	[Analysis]
[Criterion]	[Description]	[Description]	[Analysis]
[Criterion]	[Description]	[Description]	[Analysis]
[Criterion]	[Description]	[Description]	[Analysis]

9. Conclusion and Future Work

9.1 Conclusion

[Summarize the problem, proposed solution, implementation, evaluation, and main findings.]

9.2 Contributions

[Highlight the main contributions of the project from a software engineering perspective.]

9.3 Future Work

[Suggest realistic enhancements, additional features, improved architecture, further experiments, or deployment improvements.]

References

Reference List

[List all references cited in the document using the required citation style. Include books, articles, websites, documentation, standards, and datasets where applicable.]

Appendices

Appendix A: Additional Diagrams

[Place large diagrams or supporting visuals here.]

[Insert diagram / figure here]

Diagram / Figure Placeholder

Appendix B: Source Code Samples

[Place important code extracts only when necessary.]

Appendix C: User Manual / Installation Guide

[Provide detailed instructions for users or evaluators.]

Appendix D: Additional Test Cases

[Include extended test cases and evidence.]

Test Case ID	Scenario	Expected Result	Actual Result	Status
TC-001	[Insert scenario]	[Expected]	[Actual]	[Pass/Fail]
TC-002	[Insert scenario]	[Expected]	[Actual]	[Pass/Fail]
TC-003	[Insert scenario]	[Expected]	[Actual]	[Pass/Fail]
TC-004	[Insert scenario]	[Expected]	[Actual]	[Pass/Fail]

Final Submission Checklist

Cover page completed with university/faculty logos, student information, supervisor name, and submission date.

All template instructions and placeholders removed or replaced.

Problem statement is clear and aligned with objectives.

Existing solution approaches are compared and cited.

Proposed solution is justified and supported by design diagrams.

Implementation and testing evidence is included.

Discussion explains the problem status after applying the solution.

References are complete and consistently formatted.

Document language, formatting, headings, tables, figures, and page numbers are reviewed.